Baseline Comparison: VGG16 & ResNet50 vs Prototypical Network

Research: Few-Shot Learning for Cooking State Recognition in Sri Lankan Vegetables Using Prototypical Networks
Author: G.H.C. Ranasinghe | 15386652

Purpose

This notebook runs the actual baseline experiments for Section 4.2.7 of the thesis.
Both VGG16 and ResNet50 are fine-tuned on the **same 331-image dataset** and evaluated on:

1. **Standard accuracy** — full test set (53 images), all training data available
2. **3-shot accuracy** — nearest-centroid classifier using only 3 support examples per class

The same test split used in the prototypical network experiments is used here for a fair comparison.

```
# ============================================================
# Cell 1: Mount Google Drive and verify dataset
# ============================================================
from google.colab import drive
drive.mount('/content/drive')

import os

SAVE_DIR = '/content/drive/MyDrive/Research-2026/SriLankanFoodRecognition'
DATASET_SPLIT_DIR = os.path.join(SAVE_DIR, 'dataset 1')  # ✅ use 'dataset 1' folder in Drive

# Check if dataset exists
if os.path.exists(DATASET_SPLIT_DIR):
    print(f'✅ Dataset found at: {DATASET_SPLIT_DIR}')
else:
    print(f'❌ Dataset not found at: {DATASET_SPLIT_DIR}')
    print('   Please upload or extract the dataset to this folder in Drive.')

# Verify structure
for split in ['train', 'val', 'test']:
    split_path = os.path.join(DATASET_SPLIT_DIR, split)
    if os.path.exists(split_path):
        classes = [d for d in os.listdir(split_path)
                   if os.path.isdir(os.path.join(split_path, d))]
        total = sum(len(os.listdir(os.path.join(split_path, c))) for c in classes)
        print(f'  {split:6s}: {len(classes)} classes, {total} images')
    else:
        print(f'  ❌ {split} split missing in {DATASET_SPLIT_DIR}')

print(f'\n✅ Prototypical model expected at: {SAVE_DIR}/best_model.pth')
print(f'   Exists: {os.path.exists(os.path.join(SAVE_DIR, "best_model.pth"))}')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force
✅ Dataset found at: /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/dataset 1
  ❌ train split missing in /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/dataset 1
  ❌ val split missing in /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/dataset 1
  ❌ test split missing in /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/dataset 1

✅ Prototypical model expected at: /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/best_model.pth
   Exists: True
```

```
!ls "/content/drive/MyDrive/Research-2026/SriLankanFoodRecognition"
```

```
 best_model.pth              confusion_matrix.png        mobile_assets
 checkpoint_epoch_10.pth     correct_predictions.png     per_class_accuracy.png
 checkpoint_epoch_20.pth    'dataset 1'                 per_class_metrics.json
 checkpoint_epoch_30.pth     dataset.zip                 round1
 checkpoint_epoch_40.pth     exported_model              training_curves.png
 checkpoint_epoch_50.pth     full_test_results.json
 classification_report.txt   incorrect_predictions.png
```

```
print("\n📂 Inside dataset_split:")
print(os.listdir('/content/dataset_split'))

for item in os.listdir('/content/dataset_split'):
    print(item, "→", os.listdir(f'/content/dataset_split/{item}')[:5])
```

```
📂 Inside dataset_split:
```

```
['train', 'val', 'test']
train → ['pumpkin_white_curry', 'greenbeans_raw', 'greenbeans_tempered', 'pumpkin_raw', 'carrot_raw']
val → ['pumpkin_white_curry', 'greenbeans_raw', 'greenbeans_tempered', 'pumpkin_raw', 'carrot_raw']
test → ['pumpkin_white_curry', 'greenbeans_raw', 'greenbeans_tempered', 'pumpkin_raw', 'carrot_raw']
```

```
import os
import shutil
import random

# ----------------------------
# Settings
# ----------------------------
SOURCE_DIR = '/content/dataset'
SPLIT_DIR = '/content/dataset_split'

train_ratio = 0.7
val_ratio   = 0.15
test_ratio  = 0.15

random.seed(42)  # ✅ Reproducible split (important for research)

# ----------------------------
# Remove old split if exists
# ----------------------------
if os.path.exists(SPLIT_DIR):
    shutil.rmtree(SPLIT_DIR)

os.makedirs(SPLIT_DIR, exist_ok=True)

# ----------------------------
# Create split
# ----------------------------
for class_name in os.listdir(SOURCE_DIR):
    class_path = os.path.join(SOURCE_DIR, class_name)

    if not os.path.isdir(class_path) or class_name.startswith('.'):
        continue

    images = [img for img in os.listdir(class_path)
              if not img.startswith('.')]

    random.shuffle(images)

    train_end = int(len(images) * train_ratio)
    val_end   = train_end + int(len(images) * val_ratio)

    splits = {
        'train': images[:train_end],
        'val':   images[train_end:val_end],
        'test':  images[val_end:]
    }

    for split_name, split_images in splits.items():
        split_class_dir = os.path.join(SPLIT_DIR, split_name, class_name)
        os.makedirs(split_class_dir, exist_ok=True)

        for img in split_images:
            shutil.copy(
                os.path.join(class_path, img),
                os.path.join(split_class_dir, img)
            )

print("✅ Dataset split created successfully!")
```

```
✅ Dataset split created successfully!
```

```
# ============================================================
# Cell 2: Imports and device setup
# ============================================================
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, models
from PIL import Image
from tqdm import tqdm
import numpy as np
import random
import json
import matplotlib.pyplot as plt
```

```
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report
import time
from datetime import datetime

# Reproducibility — same seed as main notebook
SEED = 42
torch.manual_seed(SEED)
torch.cuda.manual_seed(SEED)
np.random.seed(SEED)
random.seed(SEED)
torch.backends.cudnn.deterministic = True

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'🖥️  Device: {DEVICE}')
if torch.cuda.is_available():
    print(f'   GPU: {torch.cuda.get_device_name(0)}')
    print(f'   Memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB')

DATASET_SPLIT_DIR = '/content/dataset_split'
SAVE_DIR = '/content/drive/MyDrive/Research-2026/SriLankanFoodRecognition'
os.makedirs(SAVE_DIR, exist_ok=True)

print('\n✅ Setup complete')
```

```
🖥️  Device: cuda
   GPU: Tesla T4
   Memory: 15.6 GB

✅ Setup complete
```

```
import os

class FoodDataset(Dataset):
    """Sri Lankan food dataset — same as main notebook"""
    def __init__(self, root_dir, transform=None):
        self.root_dir = root_dir
        self.transform = transform
        self.classes = sorted([
            d for d in os.listdir(root_dir)
            if os.path.isdir(os.path.join(root_dir, d)) and not d.startswith('.')
        ])
        self.class_to_idx = {cls: idx for idx, cls in enumerate(self.classes)}
        self.samples = []
        skipped_images = 0
        for class_name in self.classes:
            class_dir = os.path.join(root_dir, class_name)
            class_idx = self.class_to_idx[class_name]
            for img_name in os.listdir(class_dir):
                img_path = os.path.join(class_dir, img_name)
                if img_name.lower().endswith(('.png', '.jpg', '.jpeg')):
                    try:
                        # Attempt to open the image to check for corruption
                        Image.open(img_path).verify()
                        self.samples.append((img_path, class_idx))
                    except Exception as e:
                        print(f"⚠️  Skipping corrupted image: {img_path} ({e})")
                        skipped_images += 1

        print(f'✅ Loaded {len(self.samples)} images from {len(self.classes)} classes')
        if skipped_images > 0:
            print(f'   (Skipped {skipped_images} corrupted images)')
        print(f'   Classes: {self.classes}')

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        img_path, label = self.samples[idx]
        image = Image.open(img_path).convert('RGB')
        if self.transform:
            image = self.transform(image)
        return image, label


# Transforms — identical to main notebook
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.1),
    transforms.ToTensor(),
```

```
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

# Load datasets
print('Loading datasets...')
train_dataset = FoodDataset(os.path.join(DATASET_SPLIT_DIR, 'train'), transform=train_transform)
val_dataset   = FoodDataset(os.path.join(DATASET_SPLIT_DIR, 'val'),   transform=val_transform)
test_dataset  = FoodDataset(os.path.join(DATASET_SPLIT_DIR, 'test'),  transform=val_transform)

NUM_CLASSES = len(train_dataset.classes)
CLASS_NAMES = train_dataset.classes

# Standard DataLoaders for CNN training
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True,  num_workers=2, pin_memory=True)
val_loader   = DataLoader(val_dataset,   batch_size=16, shuffle=False, num_workers=2, pin_memory=True)
test_loader  = DataLoader(test_dataset,  batch_size=16, shuffle=False, num_workers=2, pin_memory=True)

print(f'\n📊 Dataset Summary:')
print(f'   Train : {len(train_dataset)} images')
print(f'   Val   : {len(val_dataset)} images')
print(f'   Test  : {len(test_dataset)} images')
print(f'   Classes ({NUM_CLASSES}): {CLASS_NAMES}')
```

```
Loading datasets...
⚠️  Skipping corrupted image: /content/dataset_split/train/pumpkin_red_curry/pumpkin_red_curry_037.jpg (cannot id
✅ Loaded 229 images from 8 classes
   (Skipped 1 corrupted images)
   Classes: ['carrot_raw', 'carrot_white_curry', 'greenbeans_raw', 'greenbeans_tempered', 'greenbeans_white_curry
✅ Loaded 48 images from 8 classes
   Classes: ['carrot_raw', 'carrot_white_curry', 'greenbeans_raw', 'greenbeans_tempered', 'greenbeans_white_curry
✅ Loaded 53 images from 8 classes
   Classes: ['carrot_raw', 'carrot_white_curry', 'greenbeans_raw', 'greenbeans_tempered', 'greenbeans_white_curry

📊 Dataset Summary:
   Train : 229 images
   Val   : 48 images
   Test  : 53 images
   Classes (8): ['carrot_raw', 'carrot_white_curry', 'greenbeans_raw', 'greenbeans_tempered', 'greenbeans_white_c
```

```
# ============================================================
# Cell 4: Training and evaluation helper functions
# ============================================================

def train_epoch(model, loader, optimizer, criterion, device):
    """One training epoch for standard CNN classifier"""
    model.train()
    total_loss, correct, total = 0.0, 0, 0
    for imgs, labels in loader:
        imgs, labels = imgs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(imgs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * imgs.size(0)
        _, predicted = outputs.max(1)
        correct += predicted.eq(labels).sum().item()
        total += labels.size(0)
    return total_loss / total, 100.0 * correct / total


def evaluate_standard(model, loader, device):
    """Standard full-dataset accuracy evaluation"""
    model.eval()
    correct, total = 0, 0
    all_preds, all_labels = [], []
    with torch.no_grad():
        for imgs, labels in loader:
            imgs, labels = imgs.to(device), labels.to(device)
            outputs = model(imgs)
            _, predicted = outputs.max(1)
            correct += predicted.eq(labels).sum().item()
            total += labels.size(0)
            all_preds.extend(predicted.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())
    accuracy = 100.0 * correct / total
```

```
    return accuracy, all_preds, all_labels


def extract_features(model, feature_extractor, img_path, transform, device):
    """Extract feature vector from a single image using CNN backbone"""
    img = Image.open(img_path).convert('RGB')
    tensor = transform(img).unsqueeze(0).to(device)
    with torch.no_grad():
        features = feature_extractor(tensor)
        features = features.squeeze()
        features = F.normalize(features, p=2, dim=0)  # L2 normalize for fair comparison
    return features.cpu()


def evaluate_k_shot(feature_extractor, dataset, transform, device, k_shot=3, n_episodes=200):
    """
    K-shot nearest-centroid evaluation.
    Mimics the few-shot protocol from the prototypical network evaluation:
    - Sample k support examples per class
    - Compute class centroid from support features
    - Classify query examples by nearest centroid (Euclidean distance)
    - Average accuracy over n_episodes

    This uses the same N_WAY=4, Q_QUERY=2 as the main notebook.
    """
    N_WAY = 4
    Q_QUERY = 2

    # Group samples by class
    class_samples = {}
    for img_path, label in dataset.samples:
        if label not in class_samples:
            class_samples[label] = []
        class_samples[label].append(img_path)

    episode_accs = []

    for _ in range(n_episodes):
        # Sample N_WAY classes
        selected_classes = random.sample(list(class_samples.keys()), N_WAY)

        support_features = []
        support_labels = []
        query_features = []
        query_labels = []

        for episode_label, class_idx in enumerate(selected_classes):
            imgs = class_samples[class_idx]
            # Need k_shot + Q_QUERY examples; skip if not enough
            if len(imgs) < k_shot + Q_QUERY:
                sampled = random.choices(imgs, k=k_shot + Q_QUERY)  # sample with replacement if needed
            else:
                sampled = random.sample(imgs, k_shot + Q_QUERY)

            support_imgs = sampled[:k_shot]
            query_imgs   = sampled[k_shot:k_shot + Q_QUERY]

       # Extract features
            for img_path in support_imgs:
                feat = extract_features(None, feature_extractor, img_path, transform, device)  # ← fixed
                support_features.append(feat)
                support_labels.append(episode_label)

            for img_path in query_imgs:
                feat = extract_features(None, feature_extractor, img_path, transform, device)  # ← fixed
                query_features.append(feat)
                query_labels.append(episode_label)

        # Compute class centroids from support set
        support_features = torch.stack(support_features)   # [N_WAY*k_shot, D]
        support_labels_t = torch.tensor(support_labels)
        query_features   = torch.stack(query_features)     # [N_WAY*Q_QUERY, D]
        query_labels_t   = torch.tensor(query_labels)

        centroids = torch.stack([
            support_features[support_labels_t == c].mean(0)
            for c in range(N_WAY)
        ])  # [N_WAY, D]

        # Nearest centroid classification
        dists = torch.cdist(query_features, centroids)  # [N_WAY*Q_QUERY, N_WAY]
        predictions = dists.argmin(dim=1)
```

```
            acc = (predictions == query_labels_t).float().mean().item() * 100.0
            episode_accs.append(acc)

    mean_acc = np.mean(episode_accs)
    std_acc  = np.std(episode_accs)
    return mean_acc, std_acc


print('✅ Helper functions defined')
```

```
✅ Helper functions defined
```

```
# ============================================================
# Cell 5: Fine-tune VGG16
# ============================================================
print('=' * 60)
print('🔵 EXPERIMENT 1: VGG16 Fine-tuning')
print('=' * 60)
print(f'Start: {datetime.now().strftime("%H:%M:%S")}')

# Load pretrained VGG16
vgg16 = models.vgg16(pretrained=True)

# Replace final classifier layer for our number of classes
vgg16.classifier[6] = nn.Linear(4096, NUM_CLASSES)
vgg16 = vgg16.to(DEVICE)

# Optimizer and loss
# Lower LR for pretrained layers, higher for new classifier head
vgg16_optimizer = optim.Adam([
    {'params': vgg16.features.parameters(), 'lr': 0.00001},  # frozen-ish
    {'params': vgg16.classifier.parameters(), 'lr': 0.0001}  # new head
])
vgg16_scheduler = optim.lr_scheduler.StepLR(vgg16_optimizer, step_size=10, gamma=0.5)
criterion = nn.CrossEntropyLoss()

VGG16_EPOCHS = 25
vgg16_train_accs = []
vgg16_val_accs   = []
best_vgg16_acc   = 0.0

for epoch in range(VGG16_EPOCHS):
    t_loss, t_acc = train_epoch(vgg16, train_loader, vgg16_optimizer, criterion, DEVICE)
    v_acc, _, _   = evaluate_standard(vgg16, val_loader, DEVICE)
    vgg16_scheduler.step()

    vgg16_train_accs.append(t_acc)
    vgg16_val_accs.append(v_acc)

    if v_acc > best_vgg16_acc:
        best_vgg16_acc = v_acc
        torch.save(vgg16.state_dict(), f'{SAVE_DIR}/best_vgg16.pth')
        marker = ' ✅ BEST'
    else:
        marker = ''

    print(f'Epoch {epoch+1:2d}/{VGG16_EPOCHS} | '
          f'Loss: {t_loss:.4f} | Train: {t_acc:.2f}% | Val: {v_acc:.2f}%{marker}')

print(f'\n🏆 Best VGG16 Val Accuracy: {best_vgg16_acc:.2f}%')
print(f'End: {datetime.now().strftime("%H:%M:%S")}')
```

```
============================================================
🔵 EXPERIMENT 1: VGG16 Fine-tuning
============================================================
Start: 15:30:46
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained'
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a wei
  warnings.warn(msg)
Epoch  1/25 | Loss: 1.4744 | Train: 48.91% | Val: 79.17% ✅ BEST
Epoch  2/25 | Loss: 0.5794 | Train: 75.98% | Val: 81.25% ✅ BEST
Epoch  3/25 | Loss: 0.3685 | Train: 85.59% | Val: 75.00%
Epoch  4/25 | Loss: 0.1774 | Train: 93.45% | Val: 77.08%
Epoch  5/25 | Loss: 0.1431 | Train: 95.20% | Val: 83.33% ✅ BEST
Epoch  6/25 | Loss: 0.1495 | Train: 93.01% | Val: 81.25%
Epoch  7/25 | Loss: 0.1012 | Train: 94.76% | Val: 83.33%
Epoch  8/25 | Loss: 0.1184 | Train: 95.63% | Val: 87.50% ✅ BEST
Epoch  9/25 | Loss: 0.0728 | Train: 97.38% | Val: 85.42%
Epoch 10/25 | Loss: 0.0940 | Train: 97.38% | Val: 85.42%
Epoch 11/25 | Loss: 0.0450 | Train: 98.69% | Val: 85.42%
Epoch 12/25 | Loss: 0.0246 | Train: 99.13% | Val: 85.42%
Epoch 13/25 | Loss: 0.0179 | Train: 99.56% | Val: 87.50%
Epoch 14/25 | Loss: 0.0378 | Train: 99.56% | Val: 87.50%
```

```
Epoch 15/25 | Loss: 0.0176 | Train: 99.13% | Val: 89.58% ✅ BEST
Epoch 16/25 | Loss: 0.0158 | Train: 99.56% | Val: 87.50%
Epoch 17/25 | Loss: 0.0183 | Train: 98.69% | Val: 87.50%
Epoch 18/25 | Loss: 0.0085 | Train: 100.00% | Val: 87.50%
Epoch 19/25 | Loss: 0.0190 | Train: 99.13% | Val: 87.50%
Epoch 20/25 | Loss: 0.0120 | Train: 99.56% | Val: 89.58%
Epoch 21/25 | Loss: 0.0589 | Train: 98.69% | Val: 87.50%
Epoch 22/25 | Loss: 0.0177 | Train: 99.56% | Val: 87.50%
Epoch 23/25 | Loss: 0.0113 | Train: 99.56% | Val: 87.50%
Epoch 24/25 | Loss: 0.0347 | Train: 98.69% | Val: 87.50%
Epoch 25/25 | Loss: 0.0038 | Train: 100.00% | Val: 87.50%

🏆 Best VGG16 Val Accuracy: 89.58%
End: 15:34:18
```

```
# ============================================================
# Cell 6: Fine-tune ResNet50
# ============================================================
print('=' * 60)
print('🟠 EXPERIMENT 2: ResNet50 Fine-tuning')
print('=' * 60)
print(f'Start: {datetime.now().strftime("%H:%M:%S")}')

# Load pretrained ResNet50
resnet50 = models.resnet50(pretrained=True)

# Replace final fully connected layer
in_features = resnet50.fc.in_features  # 2048
resnet50.fc = nn.Linear(in_features, NUM_CLASSES)
resnet50 = resnet50.to(DEVICE)

# Optimizer
resnet50_optimizer = optim.Adam([
    {'params': list(resnet50.parameters())[:-2], 'lr': 0.00001},  # backbone
    {'params': resnet50.fc.parameters(), 'lr': 0.0001}            # new head
])
resnet50_scheduler = optim.lr_scheduler.StepLR(resnet50_optimizer, step_size=10, gamma=0.5)

RESNET50_EPOCHS = 25
resnet50_train_accs = []
resnet50_val_accs   = []
best_resnet50_acc   = 0.0

for epoch in range(RESNET50_EPOCHS):
    t_loss, t_acc = train_epoch(resnet50, train_loader, resnet50_optimizer, criterion, DEVICE)
    v_acc, _, _   = evaluate_standard(resnet50, val_loader, DEVICE)
    resnet50_scheduler.step()

    resnet50_train_accs.append(t_acc)
    resnet50_val_accs.append(v_acc)

    if v_acc > best_resnet50_acc:
        best_resnet50_acc = v_acc
        torch.save(resnet50.state_dict(), f'{SAVE_DIR}/best_resnet50.pth')
        marker = ' ✅ BEST'
    else:
        marker = ''

    print(f'Epoch {epoch+1:2d}/{RESNET50_EPOCHS} | '
          f'Loss: {t_loss:.4f} | Train: {t_acc:.2f}% | Val: {v_acc:.2f}%{marker}')

print(f'\n🏆 Best ResNet50 Val Accuracy: {best_resnet50_acc:.2f}%')
print(f'End: {datetime.now().strftime("%H:%M:%S")}')
```

```
============================================================
🟠 EXPERIMENT 2: ResNet50 Fine-tuning
============================================================
Start: 15:34:18
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a wei
  warnings.warn(msg)
Epoch  1/25 | Loss: 2.0478 | Train: 17.03% | Val: 35.42% ✅ BEST
Epoch  2/25 | Loss: 1.6902 | Train: 58.08% | Val: 62.50% ✅ BEST
Epoch  3/25 | Loss: 1.4554 | Train: 74.67% | Val: 70.83% ✅ BEST
Epoch  4/25 | Loss: 1.2413 | Train: 78.17% | Val: 77.08% ✅ BEST
Epoch  5/25 | Loss: 0.9956 | Train: 86.03% | Val: 79.17% ✅ BEST
Epoch  6/25 | Loss: 0.7953 | Train: 86.90% | Val: 83.33% ✅ BEST
Epoch  7/25 | Loss: 0.6650 | Train: 89.52% | Val: 81.25%
Epoch  8/25 | Loss: 0.5699 | Train: 92.58% | Val: 83.33%
Epoch  9/25 | Loss: 0.4801 | Train: 93.01% | Val: 87.50% ✅ BEST
Epoch 10/25 | Loss: 0.4262 | Train: 89.96% | Val: 85.42%
Epoch 11/25 | Loss: 0.4280 | Train: 91.70% | Val: 85.42%
Epoch 12/25 | Loss: 0.3281 | Train: 94.32% | Val: 85.42%
Epoch 13/25 | Loss: 0.3190 | Train: 93.45% | Val: 87.50%
Epoch 14/25 | Loss: 0.3116 | Train: 95.20% | Val: 85.42%
```

```
Epoch 15/25 | Loss: 0.2918 | Train: 96.07% | Val: 87.50%
Epoch 16/25 | Loss: 0.2419 | Train: 96.94% | Val: 83.33%
Epoch 17/25 | Loss: 0.2393 | Train: 94.76% | Val: 85.42%
Epoch 18/25 | Loss: 0.2090 | Train: 96.51% | Val: 87.50%
Epoch 19/25 | Loss: 0.1860 | Train: 96.94% | Val: 87.50%
Epoch 20/25 | Loss: 0.2041 | Train: 95.20% | Val: 85.42%
Epoch 21/25 | Loss: 0.2116 | Train: 96.51% | Val: 87.50%
Epoch 22/25 | Loss: 0.1898 | Train: 95.63% | Val: 85.42%
Epoch 23/25 | Loss: 0.2207 | Train: 95.20% | Val: 85.42%
Epoch 24/25 | Loss: 0.2237 | Train: 96.51% | Val: 85.42%
Epoch 25/25 | Loss: 0.1854 | Train: 96.07% | Val: 87.50%

🏆 Best ResNet50 Val Accuracy: 87.50%
End: 15:36:45
```

```
# ============================================================
# Cell 7: Standard accuracy evaluation on test set
# Load best saved weights and evaluate on the held-out test set
# ============================================================
print('=' * 60)
print('📊 STANDARD ACCURACY — FULL TEST SET EVALUATION')
print('=' * 60)

# --- VGG16 ---
vgg16.load_state_dict(torch.load(f'{SAVE_DIR}/best_vgg16.pth'))
vgg16_test_acc, vgg16_preds, vgg16_true = evaluate_standard(vgg16, test_loader, DEVICE)
print(f'VGG16   Standard Test Accuracy: {vgg16_test_acc:.2f}%  ({int(vgg16_test_acc * len(test_dataset) / 100)}/

# --- ResNet50 ---
resnet50.load_state_dict(torch.load(f'{SAVE_DIR}/best_resnet50.pth'))
resnet50_test_acc, resnet50_preds, resnet50_true = evaluate_standard(resnet50, test_loader, DEVICE)
print(f'ResNet50 Standard Test Accuracy: {resnet50_test_acc:.2f}%  ({int(resnet50_test_acc * len(test_dataset) /

print(f'\nPrototypical Network Full Test Accuracy: 84.91%  (45/53 correct)  [from main notebook]')
print(f'Prototypical Network Validation Accuracy: 90.25%                  [from main notebook]')
```

```
============================================================
📊 STANDARD ACCURACY — FULL TEST SET EVALUATION
============================================================
VGG16   Standard Test Accuracy: 86.79%  (46/53 correct)
ResNet50 Standard Test Accuracy: 88.68%  (47/53 correct)

Prototypical Network Full Test Accuracy: 84.91%  (45/53 correct)  [from main notebook]
Prototypical Network Validation Accuracy: 90.25%                  [from main notebook]
```

```
import torch
import torch.nn as nn

# ----------------------------
# VGG16 Feature Extractor
# ----------------------------
class VGG16FeatureExtractor(nn.Module):
    def __init__(self, vgg16_model):
        super().__init__()
        self.features = vgg16_model.features
        self.avgpool = vgg16_model.avgpool
        # Take all classifier layers except final classification layer
        self.classifier = nn.Sequential(*list(vgg16_model.classifier.children())[:-1])

    def forward(self, x):
        x = self.features(x)
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x

vgg16_features = VGG16FeatureExtractor(vgg16).to(DEVICE).eval()

# ----------------------------
# ResNet50 Feature Extractor
# ----------------------------
class ResNet50FeatureExtractor(nn.Module):
    def __init__(self, resnet50_model):
        super().__init__()
        # Take all layers except final fc layer
        self.features = nn.Sequential(
            resnet50_model.conv1,
            resnet50_model.bn1,
            resnet50_model.relu,
            resnet50_model.maxpool,
            resnet50_model.layer1,
            resnet50_model.layer2,
            resnet50_model.layer3,
```

```
            resnet50_model.layer4,
            resnet50_model.avgpool,
            nn.Flatten()
        )

    def forward(self, x):
        x = self.features(x)
        return x

resnet50_features = ResNet50FeatureExtractor(resnet50).to(DEVICE).eval()
```

```
# ============================================================
# Cell 8: 3-shot accuracy evaluation
# Uses nearest-centroid classification on CNN features
# Same episodic protocol as the prototypical network evaluation
# (N_WAY=4, K_SHOT=3, Q_QUERY=2, n_episodes=200)
# ============================================================
print('=' * 60)
print('📊 3-SHOT ACCURACY EVALUATION')
print('Protocol: 200 episodes, 4-way 3-shot, 2 query per class')
print('=' * 60)

K_SHOT_EVAL = 3  # 3-shot as specified in thesis section 4.2.7
N_EPISODES  = 200

# Build feature extractors (remove classification head — use penultimate features)
# VGG16: remove classifier entirely, use adaptive pool output
vgg16_features = nn.Sequential(
    vgg16.features,
    vgg16.avgpool,
    nn.Flatten(),
    vgg16.classifier[0],  # fc1: 25088 -> 4096
    vgg16.classifier[1],  # ReLU
    vgg16.classifier[2],  # Dropout
    vgg16.classifier[3],  # fc2: 4096 -> 4096
    vgg16.classifier[4],  # ReLU
    vgg16.classifier[5],  # Dropout
    # Stop before final fc6 classification layer — use 4096-dim features
).to(DEVICE).eval()

# ResNet50: remove final fc layer, use 2048-dim global average pool features
resnet50_features = nn.Sequential(
    resnet50.conv1, resnet50.bn1, resnet50.relu, resnet50.maxpool,
    resnet50.layer1, resnet50.layer2, resnet50.layer3, resnet50.layer4,
    resnet50.avgpool,
    nn.Flatten()
    # Stop before resnet50.fc — use 2048-dim features
).to(DEVICE).eval()

print(f'\nRunning VGG16 {K_SHOT_EVAL}-shot evaluation ({N_EPISODES} episodes)...')
vgg16_kshot_acc, vgg16_kshot_std = evaluate_k_shot(
    vgg16_features, test_dataset, val_transform, DEVICE,
    k_shot=K_SHOT_EVAL, n_episodes=N_EPISODES
)
print(f'✅ VGG16   {K_SHOT_EVAL}-Shot Accuracy: {vgg16_kshot_acc:.2f}% ± {vgg16_kshot_std:.2f}%')

print(f'\nRunning ResNet50 {K_SHOT_EVAL}-shot evaluation ({N_EPISODES} episodes)...')
resnet50_kshot_acc, resnet50_kshot_std = evaluate_k_shot(
    resnet50_features, test_dataset, val_transform, DEVICE,
    k_shot=K_SHOT_EVAL, n_episodes=N_EPISODES
)
print(f'✅ ResNet50 {K_SHOT_EVAL}-Shot Accuracy: {resnet50_kshot_acc:.2f}% ± {resnet50_kshot_std:.2f}%')

print(f'\nPrototypical Network 3-Shot: evaluated separately in main notebook [Section 4.2.7]')
```

```
============================================================
📊 3-SHOT ACCURACY EVALUATION
Protocol: 200 episodes, 4-way 3-shot, 2 query per class
============================================================

Running VGG16 3-shot evaluation (200 episodes)...
✅ VGG16   3-Shot Accuracy: 92.12% ± 9.55%

Running ResNet50 3-shot evaluation (200 episodes)...
✅ ResNet50 3-Shot Accuracy: 92.31% ± 11.09%

Prototypical Network 3-Shot: evaluated separately in main notebook [Section 4.2.7]
```

```
# ============================================================
# Cell 9: Also evaluate prototypical network in 3-shot mode
# Load the trained prototypical model and run 3-shot evaluation
```

```
# so all three models are compared under identical conditions
# ============================================================
import torch.nn.functional as F

# Re-define EmbeddingNet (same as main notebook)
class EmbeddingNet(nn.Module):
    def __init__(self, embedding_dim=128):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, 3, padding=1);  self.bn1 = nn.BatchNorm2d(32);  self.pool1 = nn.MaxPool2d(
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1); self.bn2 = nn.BatchNorm2d(64);  self.pool2 = nn.MaxPool2d(
        self.conv3 = nn.Conv2d(64, 128, 3, padding=1);self.bn3 = nn.BatchNorm2d(128); self.pool3 = nn.MaxPool2d(
        self.conv4 = nn.Conv2d(128, 256, 3, padding=1);self.bn4= nn.BatchNorm2d(256); self.pool4 = nn.MaxPool2d(
        self.gap = nn.AdaptiveAvgPool2d(1)
        self.fc1 = nn.Linear(256, 256)
        self.dropout = nn.Dropout(0.3)
        self.fc2 = nn.Linear(256, embedding_dim)

    def forward(self, x):
        x = self.pool1(F.relu(self.bn1(self.conv1(x))))
        x = self.pool2(F.relu(self.bn2(self.conv2(x))))
        x = self.pool3(F.relu(self.bn3(self.conv3(x))))
        x = self.pool4(F.relu(self.bn4(self.conv4(x))))
        x = self.gap(x).view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        return F.normalize(self.fc2(x), p=2, dim=1)

class PrototypicalNetwork(nn.Module):
    def __init__(self, embedding_dim=128):
        super().__init__()
        self.embedding_net = EmbeddingNet(embedding_dim)
    def forward(self, x):
        return self.embedding_net(x)

# Load trained prototypical model
proto_model = PrototypicalNetwork(embedding_dim=128).to(DEVICE)
checkpoint = torch.load(f'{SAVE_DIR}/best_model.pth', map_location=DEVICE)
proto_model.load_state_dict(checkpoint['model_state_dict'])
proto_model.eval()
print(f'✅ Prototypical model loaded (epoch {checkpoint["epoch"]+1}, val acc {checkpoint["val_acc"]:.2f}%)')

# Use prototypical model as its own feature extractor for the k-shot eval
print(f'\nRunning Prototypical Network {K_SHOT_EVAL}-shot evaluation ({N_EPISODES} episodes)...')
proto_kshot_acc, proto_kshot_std = evaluate_k_shot(
    proto_model, test_dataset, val_transform, DEVICE,
    k_shot=K_SHOT_EVAL, n_episodes=N_EPISODES
)
print(f'✅ Prototypical Network {K_SHOT_EVAL}-Shot Accuracy: {proto_kshot_acc:.2f}% ± {proto_kshot_std:.2f}%')
```

```
✅ Prototypical model loaded (epoch 52, val acc 90.25%)

Running Prototypical Network 3-shot evaluation (200 episodes)...
✅ Prototypical Network 3-Shot Accuracy: 93.56% ± 11.04%
```

```
# ============================================================
# Cell 10: FINAL RESULTS SUMMARY
# ============================================================
print('\n' + '=' * 65)
print('🏆 FINAL COMPARISON RESULTS — Section 4.2.7')
print('=' * 65)
print(f'{"Model":<25} {"Standard Acc":>15} {"3-Shot Acc":>15}')
print('-' * 65)
print(f'{"VGG16 Baseline":<25} {vgg16_test_acc:>14.2f}% {vgg16_kshot_acc:>14.2f}%')
print(f'{"ResNet50 Baseline":<25} {resnet50_test_acc:>14.2f}% {resnet50_kshot_acc:>14.2f}%')
print(f'{"Prototypical Network":<25} {"84.91%":>15} {proto_kshot_acc:>14.2f}%')
print('=' * 65)
print(f'\nNotes:')
print(f'  Standard Acc  = full test set (53 images), all training data available')
print(f'  3-Shot Acc    = 200 episodes, 4-way 3-shot, 2 query per class, nearest centroid')
print(f'  Prototypical Standard Acc = 84.91% from main notebook (full test set)')
print(f'\n  VGG16   3-Shot: {vgg16_kshot_acc:.2f}% ± {vgg16_kshot_std:.2f}%')
print(f'  ResNet50 3-Shot: {resnet50_kshot_acc:.2f}% ± {resnet50_kshot_std:.2f}%')
print(f'  Proto    3-Shot: {proto_kshot_acc:.2f}% ± {proto_kshot_std:.2f}%')

# Save results to JSON for thesis documentation
results = {
    'experiment': 'Baseline Comparison — Section 4.2.7',
    'dataset': '331 images, 8 classes, same train/val/test split as main experiment',
    'evaluation_protocol': {
        'standard': 'Full test set (53 images), all training data available',
        'k_shot': f'200 episodes, 4-way {K_SHOT_EVAL}-shot, 2 query per class, nearest centroid on CNN features'
    },
```

```
    'vgg16': {
        'standard_accuracy': round(vgg16_test_acc, 2),
        'kshot_accuracy': round(vgg16_kshot_acc, 2),
        'kshot_std': round(vgg16_kshot_std, 2),
        'training_epochs': VGG16_EPOCHS,
        'optimizer': 'Adam (backbone lr=1e-5, head lr=1e-4)',
        'scheduler': 'StepLR(step_size=10, gamma=0.5)'
    },
    'resnet50': {
        'standard_accuracy': round(resnet50_test_acc, 2),
        'kshot_accuracy': round(resnet50_kshot_acc, 2),
        'kshot_std': round(resnet50_kshot_std, 2),
        'training_epochs': RESNET50_EPOCHS,
        'optimizer': 'Adam (backbone lr=1e-5, head lr=1e-4)',
        'scheduler': 'StepLR(step_size=10, gamma=0.5)'
    },
    'prototypical_network': {
        'standard_accuracy': 84.91,
        'validation_accuracy': 90.25,
        'kshot_accuracy': round(proto_kshot_acc, 2),
        'kshot_std': round(proto_kshot_std, 2),
        'source': 'main notebook V1_SriLankanFoodRecognition.ipynb'
    }
}

results_path = f'{SAVE_DIR}/baseline_comparison_results.json'
with open(results_path, 'w') as f:
    json.dump(results, f, indent=2)
print(f'\n✅ Results saved to: {results_path}')
```

```
================================================================
🏆 FINAL COMPARISON RESULTS — Section 4.2.7
================================================================
Model                          Standard Acc      3-Shot Acc
----------------------------------------------------------------
VGG16 Baseline                       86.79%          92.12%
ResNet50 Baseline                    88.68%          92.31%
Prototypical Network                 84.91%          93.56%
================================================================

Notes:
  Standard Acc  = full test set (53 images), all training data available
  3-Shot Acc    = 200 episodes, 4-way 3-shot, 2 query per class, nearest centroid
  Prototypical Standard Acc = 84.91% from main notebook (full test set)

  VGG16   3-Shot: 92.12% ± 9.55%
  ResNet50 3-Shot: 92.31% ± 11.09%
  Proto    3-Shot: 93.56% ± 11.04%

✅ Results saved to: /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/baseline_comparison_results.js
```

```
# ============================================================
# Cell 11: Visualizations
# 1) Training curves for both baselines
# 2) Comparison bar chart for thesis
# ============================================================
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# --- VGG16 training curve ---
axes[0].plot(range(1, VGG16_EPOCHS+1), vgg16_train_accs, 'b-o', markersize=3, label='Train')
axes[0].plot(range(1, VGG16_EPOCHS+1), vgg16_val_accs,   'r-s', markersize=3, label='Val')
axes[0].set_title('VGG16 Training Curves', fontsize=13, fontweight='bold')
axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Accuracy (%)')
axes[0].legend(); axes[0].grid(True, alpha=0.3)
axes[0].set_ylim(0, 105)

# --- ResNet50 training curve ---
axes[1].plot(range(1, RESNET50_EPOCHS+1), resnet50_train_accs, 'b-o', markersize=3, label='Train')
axes[1].plot(range(1, RESNET50_EPOCHS+1), resnet50_val_accs,   'r-s', markersize=3, label='Val')
axes[1].set_title('ResNet50 Training Curves', fontsize=13, fontweight='bold')
axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('Accuracy (%)')
axes[1].legend(); axes[1].grid(True, alpha=0.3)
axes[1].set_ylim(0, 105)

# --- Comparison bar chart ---
models_names = ['VGG16\nBaseline', 'ResNet50\nBaseline', 'Prototypical\nNetwork']
std_accs     = [vgg16_test_acc, resnet50_test_acc, 84.91]
kshot_accs   = [vgg16_kshot_acc, resnet50_kshot_acc, proto_kshot_acc]

x = np.arange(len(models_names))
width = 0.35
colors_std   = ['#90CAF9', '#A5D6A7', '#FFCC80']
```

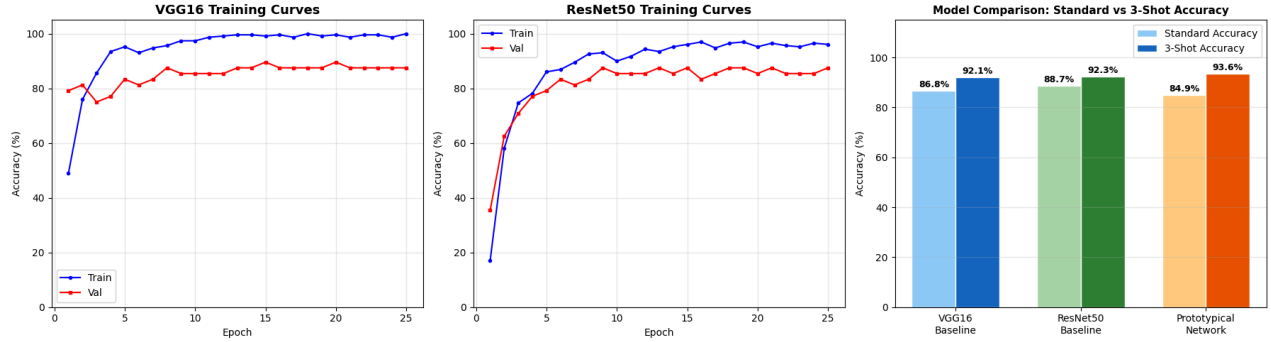
```
colors_kshot = ['#1565C0', '#2E7D32', '#E65100']

bars1 = axes[2].bar(x - width/2, std_accs,   width, label='Standard Accuracy', color=colors_std,   edgecolor='wh
bars2 = axes[2].bar(x + width/2, kshot_accs, width, label=f'{K_SHOT_EVAL}-Shot Accuracy',  color=colors_kshot, e

for bar in bars1:
    axes[2].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
                 f'{bar.get_height():.1f}%', ha='center', va='bottom', fontsize=9, fontweight='bold')
for bar in bars2:
    axes[2].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
                 f'{bar.get_height():.1f}%', ha='center', va='bottom', fontsize=9, fontweight='bold')

axes[2].set_title('Model Comparison: Standard vs 3-Shot Accuracy', fontsize=11, fontweight='bold')
axes[2].set_xticks(x); axes[2].set_xticklabels(models_names)
axes[2].set_ylabel('Accuracy (%)'); axes[2].set_ylim(0, 115)
axes[2].legend(); axes[2].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plot_path = f'{SAVE_DIR}/baseline_comparison_chart.png'
plt.savefig(plot_path, dpi=150, bbox_inches='tight')
plt.show()
print(f'✅ Chart saved to: {plot_path}')
```




```
✅ Chart saved to: /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/baseline_comparison_chart.png
```

```
# ============================================================
# Cell 12: Update thesis text with actual numbers

# ============================================================
proto_advantage_std   = 84.91 - max(vgg16_test_acc, resnet50_test_acc)
proto_advantage_kshot = proto_kshot_acc - max(vgg16_kshot_acc, resnet50_kshot_acc)
best_baseline_name    = 'ResNet50' if resnet50_test_acc > vgg16_test_acc else 'VGG16'
best_baseline_std     = max(vgg16_test_acc, resnet50_test_acc)
best_baseline_kshot   = max(vgg16_kshot_acc, resnet50_kshot_acc)


print(f"""
TABLE :

┌──────────────────────┬──────────────────────┬──────────────────────┐
│ Model                │ Standard Accuracy    │ 3-Shot Accuracy      │
├──────────────────────┼──────────────────────┼──────────────────────┤
│ VGG16 Baseline       │ {vgg16_test_acc:.2f}%          │ {vgg16_kshot_acc:.2f}%          │
│ ResNet50 Baseline    │ {resnet50_test_acc:.2f}%          │ {resnet50_kshot_acc:.2f}%          │
│ Prototypical Network │ 84.91%               │ {proto_kshot_acc:.2f}%          │
└──────────────────────┴──────────────────────┴──────────────────────┘


NARRATIVE UPDATE:
- Standard acc: proto ({84.91:.2f}%) vs best baseline {best_baseline_name} ({best_baseline_std:.2f}%)
  → Advantage: {proto_advantage_std:+.2f} percentage points
- 3-shot acc: proto ({proto_kshot_acc:.2f}%) vs best baseline ({best_baseline_kshot:.2f}%)
  → Advantage: {proto_advantage_kshot:+.2f} percentage points
""")

print('\n✅ All baseline experiments complete.')
```

TABLE :

Model	Standard Accuracy	3-Shot Accuracy
VGG16 Baseline	86.79%	92.12%
ResNet50 Baseline	88.68%	92.31%
Prototypical Network	84.91%	93.56%

NARRATIVE UPDATE:
- Standard acc: proto (84.91%) vs best baseline ResNet50 (88.68%)
  → Advantage: -3.77 percentage points
- 3-shot acc: proto (93.56%) vs best baseline (92.31%)
  → Advantage: +1.25 percentage points

✅ All baseline experiments complete.